



Lani

Your Personal Shopping Retriever — an AI shopping assistant that saves every product you look at, learns your taste, and helps you decide what to buy.

Project technical overview & demo brief

Prepared 31 August 2026

Scope: product, feature set, codebase architecture, and the engineering decisions behind them

Repository: `savemoneyai-monorepo` (main branch) · three client apps + Supabase backend

Contents

- 01** Executive summary
- 02** The product — vision & positioning
- 03** System at a glance — the three surfaces
- 04** Feature walkthrough
- 05** Architecture & codebase
- 06** Data model
- 07** Key engineering decisions & rationale
- 08** Security & privacy model
- 09** Known limitations & planned follow-ups
- 10** Technology stack reference
- 11** Suggested demo script

1 · Executive summary

Lani is an AI shopping assistant delivered across three surfaces that share one backend:

- **A Chrome extension** that quietly watches the pages you browse, decides which are real product pages, and saves them to your account automatically.
- **A website** (Next.js) where you browse everything you've saved, organise it into lists, build a cart, track prices, and chat with the assistant.
- **A mobile app** (React Native / Expo) that mirrors the website's authenticated experience on iOS and Android.

The backend is **Supabase** — Postgres with row-level security, hosted auth, file storage, scheduled jobs, and edge functions. There is deliberately almost no custom server: the three clients talk directly to the database, and Row-Level Security is the authorization layer.

The assistant is **bring-your-own-model**: each user configures Claude, OpenRouter, or a local Ollama instance with their own API key. The operator pays no per-token AI cost.

What makes it distinctive

Passive capture. You don't clip products — Lani detects them. An LLM classifies each page you visit as "product / not product" before anything is saved.

Agentic chat. The assistant doesn't just talk — it can add items to your cart, favorite them, or file them into lists, by emitting parseable action tags in its reply.

One data model, three clients. The extension, website, and app all reuse the same `lib/` conventions and the same RLS-protected tables.

Deduplicated price tracking. Everyone tracking the same URL shares one row, so the daily re-scrape scales with distinct products, not users.

2 · The product — vision & positioning

"Loyal to your taste, never forgets."

Lani is framed around a mascot — a golden retriever — and the tagline *"Your Personal Shopping Retriever."* The three product promises on the landing page are:

- **Fetches every product** — auto-saves every product page you visit, even screenshots.
- **Does helpful tricks** — price tracking, finding alternatives, styling advice.
- **Loyal to your taste** — learns what you love and personalises recommendations.

The problem it targets

Online shopping research is scattered across dozens of tabs, wishlists, screenshots, and "I'll remember that later" moments that never resurface. Existing wishlist tools require deliberate clipping and live on a single retailer. Lani's bet is that **the capture step should be invisible** — if saving is free, your entire consideration set becomes searchable, comparable, and available to an assistant that knows your history.

Who it's for

Consumers who shop across many stores and want a single organised view, plus a secondary audience of **resellers** — hence the eBay / Etsy / Reverb integrations that pull in selling and order history alongside saved products.

Naming lineage

The codebase evolved from an earlier project called **SaveMoneyAI** (still the npm package names and the salvaged colour palette). It was rebranded to **Lani**. Early database migration comments refer to the assistant as "Nora"; the current system prompt names it "Lani".

3 · System at a glance — the three surfaces

Monorepo layout

```
Lani/
├── apps/
│   ├── website/    Next.js 14 (App Router) – marketing site + authed web app
│   ├── extension/  Chrome MV3 extension (Vite build, React side panel)
│   └── app/        Expo SDK 57 / React Native 0.86 mobile app
└── apps/website/supabase/  schema, migrations, edge functions
```

`apps/website` and `apps/extension` are npm workspaces from the repo root. `apps/app` (Expo) is managed independently.

Responsibilities

Surface	Job	Stack
Chrome extension v0.1.0, on the Chrome Web Store	Passive product detection & capture while browsing; local capture queue; side-panel UI for sign-in, captured items, activity log, blacklist, and AI settings.	Manifest V3, TypeScript, React (side panel), Vite, Tailwind, <code>@supabase/supabase-js</code>
Website port 3001, Docker	Public landing pages; authenticated app: dashboard, chat, cart, lists, product detail, profile, public shared-list pages; legal/marketing pages; a thin API layer for the AI proxy and OAuth.	Next.js 14 App Router, React 18, TypeScript, Tailwind, standalone output
Mobile app EAS project <code>@bbanothu/lani</code>	Authenticated experience on iOS/Android: dashboard, chat, cart, lists + detail, profile with avatar picker.	Expo SDK 57, React Native 0.86, React 19, AsyncStorage, expo-image-picker, expo-blur
Supabase project ref <code>uxroei...</code>	Postgres + RLS (the authorization layer), hosted Auth, Storage (avatars), Edge Functions (account deletion, price re-scrape), <code>pg_cron</code> + <code>pg_net</code> (daily job), Realtime (live UI refresh).	Supabase managed platform

How the surfaces connect

- All three clients authenticate against the same Supabase Auth project and read/write the same tables under RLS.
- The extension captures locally, then a background worker syncs the queue straight into the `products` table.

- The website and app render from those tables and subscribe to Realtime changes, so a capture from the extension appears on an open dashboard within seconds.
- A small content script (`bridge.js`) runs only on the website's own origin so the site can ask the extension to open many cart tabs at once (browsers block multi- `window.open` from a page).

4 · Feature walkthrough

4.1 Automatic product capture EXTENSION

A content script runs on every page at `document_idle`. The decision pipeline:

1. **Guard checks** — bail if the extension is paused or the user isn't signed in.
2. **Known-retailer fast path** — for Amazon, eBay, Etsy, Walmart, Target, and Best Buy, a regex tests whether the URL matches that site's product-page shape (e.g. `/dp/<ASIN>`, `/itm/`, `/listing/<id>`). A confident match captures immediately, with no AI call.
3. **Domain blacklist** — for other sites, if the AI previously answered "not a product page" for this domain, skip silently.
4. **AI classification** — otherwise, send the page title, URL, and first 1,500 characters of text to the user's configured model: *"Is this a single product page? Reply YES or NO, then 3–5 tags."*
5. **Extraction** — `extractProductInfo()` reads `og: / product:price` meta tags (with Amazon-specific DOM selectors as a fallback) to get title, price, currency, image, URL, domain.
6. **Queue** — the item goes into a local "outbox" in `chrome.storage`; a per-URL "seen" set prevents duplicates on revisit.

A background worker drains the outbox to Supabase every 5 seconds, with a `chrome.alarms` 1-minute backstop (MV3 suspends idle workers). The side panel shows a live activity log, the pending queue, the domain blacklist (editable), and a global pause switch.

4.2 Product dashboard WEB MOBILE

- Grid of every saved product, newest first, refreshed live via a Realtime subscription.
- **Auto-generated filter chips** — the client tallies frequent words across product titles (minus stop-words) and offers the top 8 as one-tap filters; users can add their own keyword chips.
- **Retailer rail** — filter to a single store.
- Sort by date / name / price, ascending or descending.

4.3 Product detail WEB

- Large image, price, and quick toggles: favorite, add-to-cart, and **track price**.
- Price-history view for tracked items.
- A multi-store price comparison block (currently populated with seeded mock figures — see limitations).
- Delete offers a choice: just remove it, or also **blacklist the domain** so nothing from that store is captured again.

4.4 Lists & Favorites WEB MOBILE

- Create any number of lists, each `private` / `shared` / `public`.
- A **Favorites** list is auto-created for every new user by a database trigger and cannot be deleted (a second trigger blocks it).
- Share a list by email; invitees see it under a "**Shared with me**" tab.
- Public / shared lists have a read-only page at `/s/<id>` viewable by anyone, including logged-out visitors.

4.5 Cart WEB MOBILE

Add / remove / clear, and "**open all in tabs**" — which routes through the extension bridge when present (a page can't fire many `window.open` calls past a popup blocker, but the extension can).

4.6 AI chat assistant WEB MOBILE

A streaming chat with "Lani". Key mechanics:

- **Grounding.** Every request injects a system message listing the user's saved products (numbered, with price / store / saved-status) and their list names. The model is told to prefer items from this list and to say so when something's missing.
- **Product cards.** The model ends a reply with a line like `[[products:1,3]]`; the client strips that tag and renders those items as a scrollable card rail beneath the message.
- **Agentic action tags.** The model can embed `[[action:add_cart:2]]`, `[[action:favorite:2]]`, `[[action:unfavorite:2]]`, or `[[action:add_list:2:Winter]]` anywhere in its reply. The client parses these out (they're hidden from the displayed text) and runs the real `lib/` mutations. This gives tool-like behaviour that works identically across Claude, OpenRouter models, and local Ollama — none of which need to support a function-calling API.
- **Voice input** via the browser Web Speech API.
- **History.** Sessions and messages persist to `chat_sessions` / `chat_messages`; a history browser lets you reopen and continue past chats. A trigger bumps `updated_at` so the list stays ordered by recent activity without a join.
- **Transport.** On web, Claude / OpenRouter calls go through a Next API route (`/api/chat`) that re-streams Server-Sent Events; Ollama calls go straight from the browser (a deployed server can't reach `localhost`). On mobile, all providers are called directly with a streaming `XMLHttpRequest`.

4.7 Price tracking WEB CRON

- Tracking a product calls a `security_definer` function that finds-or-creates a shared `tracked_products` row keyed by URL, links the product to it, and seeds `price_history` with the current price.
- A `pg_cron` job at 08:00 UTC daily calls `pg_net.http_post` to wake the `rescrape-prices` Edge Function.
- The function (service-role) fetches each distinct tracked URL, regex-extracts a price from the static HTML, and on a change writes a new `price_history` row and propagates the price to every linked `products` row.

4.8 Marketplace integrations WEB

Connect **eBay**, **Etsy**, or **Reverb** via OAuth to pull selling / order history into the profile. Each provider has three API routes — `start` (build the authorize URL), `callback` (exchange the code, store tokens), `orders` (fetch, refreshing the token first if expired). The verified user id is carried through the third-party redirect in an **HMAC-signed state parameter** (Etsy also carries a PKCE verifier). Tokens live in the `integrations` table under RLS.

4.9 Profile & account WEB MOBILE

- Avatar upload to a public Storage bucket, scoped so a user can only write into their own `{user_id}/` folder.
- Editable display name.
- AI provider settings: provider, API key, model, Ollama base URL — stored client-side only.
- "Top stores" breakdown of where saved products come from.
- **Delete account** — an Edge Function deletes the auth user; every user table has `on delete cascade`, so Postgres removes the rest.

4.10 Bring-your-own AI model ALL

Three providers everywhere: **Claude**, **OpenRouter** (any model it proxies), and **Ollama** (local, default `llama3.1`). Keys are held in `localStorage` (web), `AsyncStorage` (mobile), or `chrome.storage` (extension) — never on an operator server.

5 · Architecture & codebase

5.1 The "backend is the database" approach

There is no application server for ordinary reads and writes. Each client uses

`@supabase/supabase-js` with the public anon key; **every table has RLS policies** of the form `user_id = auth.uid()`, so a client physically cannot read another user's rows. Server code exists only in three narrow places:

- **Next API routes** — the AI proxy (`/api/chat`) and the nine OAuth routes, which need provider secrets.
- **Supabase Edge Functions** — `delete-account` and `rescrape-prices`, which need the service-role key.
- **Database functions / triggers** — invariants that must hold regardless of client (Favorites list, price tracking, chat-session ordering).

5.2 Extension internals

```
src/
├── content/detect.ts    runs in every page: the capture decision pipeline
├── lib/extract.ts       DOM/meta scraping (serialized by source text into the page)
├── lib/site-patterns.ts per-retailer product-URL regexes + domain-label matcher
├── lib/llm.ts           Claude / OpenAI-compatible / Ollama chat calls
├── lib/storage.ts       chrome.storage wrappers: settings, outbox, blacklist,
                        seen-URLs, logs, pause flag
├── lib/supabase.ts      one Supabase client, owns the session
├── background/index.ts  service worker: outbox→Supabase sync, all network LLM
                        calls, auth message handlers, dev auto-reload
└── sidepanel/SidePanel.tsx 632-line React side panel (auth gate, items, logs,
                        blacklist editor, AI settings)
```

Three Vite build configs produce the side panel, the content script, and the background worker separately.

5.3 Website structure

- **App Router**, almost entirely client components (`'use client'`) because the data layer is the browser Supabase client.
- `app/page.tsx` redirects to `/dashboard` or `/home` based on session.
- `components/figma-landing/*` — the marketing landing page, built section-by-section from a Figma design (hero, bento grid, testimonials carousel, how-it-works, CTA).
- `lib/*` — one module per domain object (`products`, `lists`, `cart`, `chat-history`, `integrations`, `auth`, ...), each exposing typed functions and a `subscribeTo...` Realtime

helper.

- `lib/popups.tsx` — a small queue-based replacement for `window.alert/confirm/prompt` with custom modals (list picker, share dialog, delete-choice).
- Built with `output: 'standalone'` and `outputFileTracingRoot` pointed at the repo root, then packaged by a multi-stage `Dockerfile` that builds from the monorepo root (the lockfile covers all workspaces).

5.4 Mobile app structure

A deliberately simple **state-machine navigation** in `App.tsx` — a `tab` state variable swaps between `Login`, `Dashboard`, `Cart`, `Chat`, `Lists`, `Profile` (no navigation library). The `lib/` modules mirror the website's, re-pointed at `AsyncStorage` and native `fetch/XHR`. `AGENTS.md` pins the team to the exact Expo SDK 57 docs because the SDK changed significantly.

6 · Data model

Table	Holds	Notable rules
products	Every saved product (title, price, currency, image, url, domain, source, tags).	RLS: owner only. <code>source</code> ∈ {extension, manual}. Later columns: <code>tracking</code> , <code>tracked_product_id</code> .
lists	User-created lists.	Owner-only writes; public read for shared / public . One <code>is_favorites</code> list per user (partial unique index); protected from deletion by trigger.
list_products	List ↔ product join.	Access via the owning list; public read for shared/public lists.
cart_items	The cart.	Owner only; unique on (user, product).
chat_sessions / chat_messages	Persisted assistant conversations; messages carry <code>product_ids[]</code> .	Owner only; a trigger touches <code>updated_at</code> on each new message.
tracked_products	Shared, URL-deduplicated price-tracking rows.	Readable by any signed-in user; written only by <code>security_definer</code> functions and the service-role job.
price_history	Time series of observed prices.	Readable only for products the user tracks.
integrations	OAuth tokens for eBay / Etsy / Reverb.	Owner only; unique on (user, provider).
blacklisted_domains	Domains a user flagged while deleting a product.	Owner only. Written by the website; not yet consulted by the extension.
list_shares	Emails a list is shared with.	Owner manages; an invitee can read rows matching their own JWT email.
storage: avatars	Profile pictures.	Public read; write restricted to <code>{uid}/</code> path.

Migration timeline

Nine migrations spanning 15–21 Aug 2026: initial schema → chat history → price tracking (+ `pg_cron`) → integrations (eBay) → blacklisted domains → Etsy → list sharing → Reverb → avatar storage. The feature set grew roughly one capability every day or two.

Database functions & triggers

- `handle_new_user()` — after auth signup, inserts the user's Favorites list.
- `protect_favorites_list()` — `before delete` on lists, raises if `is_favorites` .
- `touch_chat_session()` — keeps session ordering fresh without a read-time join.
- `track_product()` / `untrack_product()` — the deduplicating tracker, `security definer` so it can write the shared table.

7 · Key engineering decisions & rationale

7.1 RLS-first, near-serverless backend

Decision: no bespoke CRUD API; clients hit Postgres directly; RLS is the security boundary.

WHY

Three clients (extension, web, mobile) would otherwise each need an API client, and the operator would need to build, host, and secure a REST layer that mostly just re-implements `SELECT ... WHERE user_id = me`. Pushing that predicate into RLS policies means the rule is enforced once, in the database, for every client and every query path. Server code is then reserved for the two things RLS can't do: hold secrets, and act across users.

7.2 Decoupled capture and sync (the outbox pattern)

Decision: the content script only queues captures into local storage; a separate background worker drains that queue to Supabase on a 5-second interval plus a 1-minute alarm.

WHY

Page detection must be instant and must work offline. If capture wrote to the network inline, a slow or failed request would stall detection on the next page. Queuing locally is cheap and reliable; the alarm exists because Manifest V3 suspends idle service workers, which would silently kill a bare `setInterval`. On failure the queue is simply left for the next tick.

7.3 LLM calls run in the background worker / server, not the page

Decision: the extension's content script never calls a model directly — it messages the background worker. On the website, Claude/OpenRouter go through `/api/chat`; only Ollama is called from the browser.

WHY

A content script's `fetch` runs in the visited page's security context, so an `https://` page cannot reach a plain `http://localhost:11434` Ollama endpoint (blocked as mixed content). The background worker isn't bound to any page's protocol, so it can. On the web, the server proxy keeps SSE handling uniform and avoids per-provider CORS quirks. Ollama is the exception everywhere — it lives on the user's own machine, which a deployed server can never reach, so that request has to originate in the browser.

7.4 Bring-your-own API key

Decision: users supply their own Claude / OpenRouter key, or run Ollama locally. Keys are stored only on the client.

WHY

AI inference is the dominant variable cost of a product like this. BYO-key moves that cost to the user and removes the operator's need for metering, quotas, and abuse defence. The local-Ollama option makes the product usable at zero marginal cost and with no data leaving the machine.

7.5 Known-retailer URL regex vs. AI classification vs. domain blacklist

Decision: a three-tier funnel — hard-coded product-URL patterns for six big retailers, then a per-domain "not a product site" blacklist, then the LLM.

WHY

Big retailers mix product, search, category, and cart pages under one domain, so a single "no" answer must *not* blacklist the whole domain — instead their known URL shapes are matched directly, which is also more reliable than a weak local model's guess. For a site Lani has never seen, one "no" *does* blacklist the domain, to avoid paying for an AI call on every future page load there. A URL that doesn't match a known pattern still falls through to the AI rather than being auto-rejected, because the regex set is deliberately incomplete.

7.6 Tolerating weak local models

Decision: the classifier prompt asks for "YES/NO on line one", but the parser strips leading non-letters, accepts `**YES**` / `Answer: yes`, caps page text at 1,500 chars, and runs at temperature 0.3.

WHY

A 7–8B local model frequently disobeys strict formatting instructions. Rather than fail closed on formatting noise, the parser normalises aggressively so the common disobedient shapes still resolve correctly.

7.7 Deduplicated `tracked_products`

Decision: price tracking rows are keyed by URL and shared across all users who track that URL; user-specific state is just a boolean + FK on the user's own `products` row.

WHY

The daily re-scrape cost scales with the number of *distinct* product URLs, not with users × products. Ten users tracking the same popular item cause one fetch per day, not ten.

7.8 HMAC-signed OAuth state instead of a session through the redirect

Decision: the `state` parameter is a base64url payload (`{uid, exp}` , 10-minute TTL) plus an HMAC signature using the provider's client secret; the callback verifies it with a timing-safe compare and then writes the token row with the service-role client.

WHY

The third-party redirect lands on a route with no cookie / session for that user. Rather than depend on cross-site cookies surviving the round trip, the verified user id is carried in the state itself, signed so the callback can trust it and time-boxed so a leaked URL is useless after ten minutes. Etsy's flow additionally tucks the PKCE verifier into the same signed blob.

7.9 Chat action tags instead of function calling

Decision: the assistant performs actions by emitting text tags like `[[action:add_cart:2]]` , parsed and executed client-side.

WHY

Function/tool calling has a different API shape for every provider, and local Ollama models often don't support it well. A plain-text tag convention works identically on Claude, any OpenRouter model, and Ollama, needs no schema negotiation, and degrades gracefully — a model that ignores the convention just chats normally.

7.10 Invariants enforced by database triggers

Decision: the Favorites list is created and protected by triggers; chat-session ordering is maintained by a trigger.

WHY

With three independent clients writing directly to the tables, an invariant enforced in client code would only be as strong as the least careful client. Triggers make "every user has exactly one Favorites list, and it can't be deleted" true no matter who's calling.

7.11 Realtime refetch, not Realtime payloads

Decision: a Realtime event just calls `getProducts()` again through the normal RLS-scoped query; the event payload itself is never read.

WHY

It's simpler and safe: the refetch is already correctly filtered per user, so it doesn't matter that the raw change feed isn't. Slight extra bandwidth in exchange for no payload-trust logic.

7.12 Next standalone output + monorepo tracing root

Decision: `output: 'standalone'` with `outputFileTracingRoot` at the repo root; Docker builds from the monorepo root.

WHY

npm workspaces hoist `node_modules` to the root, so Next's file tracing has to look there to assemble a correct standalone bundle. Building from the root also lets the single lockfile drive `npm ci`.

8 · Security & privacy model

- **Row-Level Security on every user table** — the anon key alone grants a client nothing beyond its own rows. Shared/public list policies check the list's own `visibility` column, deliberately not `auth.vid()`, so logged-out visitors can view a public list.
- **Service-role key** appears only in the OAuth callback routes and the two Edge Functions — never shipped to a client.
- **OAuth state** is signed (HMAC-SHA256), time-boxed (10 min), and compared with `timingSafeEqual`.
- **Storage** — the avatars bucket is world-readable (they're just images) but write policies pin uploads to `{uid}/`.
- **Account deletion** is real and complete: the auth user is removed and all data cascades via foreign keys.
- **AI keys** never reach an operator server, with one caveat: on the website, Claude/OpenRouter requests pass *through* `/api/chat` (the key is forwarded upstream, not stored). The extension and mobile app call those providers directly.
- **Extension surface area** — `host_permissions: <all_urls>` and it reads up to 1,500 characters of page text to classify pages, sending that excerpt to the user's chosen model. The setup page is explicit that Lani "never saves without your say-so".

9 · Known limitations & planned follow-ups

These are called out honestly in code comments — useful to acknowledge in a demo before someone finds them.

Area	Current state	Planned direction
Domain blacklist	<code>blacklisted_domains</code> is written by the website's delete flow but the extension's capture path doesn't consult it yet.	Wire the content script to skip blacklisted domains.
Price re-scrape coverage	Plain <code>fetch</code> + regex only gets a price from sites that render it in static server HTML. JS-rendered pricing and bot-walled sites (Amazon blocks non-browser fetches) return nothing.	Headless-browser fetch (Browserless / ScrapingBee) for domains that come back empty.
Reverb integration	OAuth URLs, token request shape, and API paths are best-effort from sparse public docs.	Verify against a real approved Reverb app; the orders endpoint path in particular may change.
Multi-store price comparison	The block on the product detail page uses seeded pseudo-random numbers, not real competitor prices.	Replace with real comparison data (or remove until available).
Model id drift	Chat defaults reference <code>claude-sonnet-4-20250514</code> in the web/mobile chat paths while the extension defaults to <code>claude-sonnet-5</code> .	Normalise the default model id across surfaces.
Repo hygiene	<code>README.md</code> is a stub; the most recent commits are squash-style ("temp", "ui", "App").	Fill in the README; tidy history before any external handoff.

10 · Technology stack reference

Layer	Choice
Language	TypeScript throughout
Website	Next.js 14.2 (App Router), React 18.3, Tailwind CSS 3.4, Nunito font
Extension	Chrome Manifest V3, React 18.3 side panel, Vite 5.2 (three build configs), Tailwind
Mobile	Expo SDK 57, React Native 0.86, React 19.2, AsyncStorage, expo-image-picker, expo-blur, react-native-safe-area-context
Backend	Supabase — Postgres, Auth, Storage, Edge Functions (Deno), Realtime, <code>pg_cron</code> + <code>pg_net</code>
Data access	<code>@supabase/supabase-js</code> v2 on every client; RLS policies as the authz layer
AI providers	Anthropic Messages API, OpenRouter, Ollama (OpenAI-compatible) — all streaming (SSE)
Integrations	eBay Sell Fulfillment API, Etsy Open API v3 (PKCE), Reverb API
Deploy	Website: multi-stage Docker (<code>node:20-alpine</code> , standalone). Mobile: EAS Build (iOS build 5, Android versionCode 4). Extension: Chrome Web Store.
Formatting	Prettier (repo-wide <code>format</code> script)

Repository facts

- npm workspaces: `apps/website` , `apps/extension` ; `apps/app` standalone.
- Supabase project ref `uxroeiaomhjwxmaehami` ; EAS project id `e12f5543-...` ; bundle id `com.brainrotslop.lani` .
- Chrome Web Store listing id `ekcgdpkgpgohmogglfceiibfgpcppefh` .

11 · Suggested demo script

1. **Frame the problem (30s).** "Shopping research scatters across tabs and screenshots. Lani makes saving invisible so your whole consideration set is searchable."
2. **Show passive capture (90s).** With the extension installed and signed in, browse to a product page on a couple of different stores. Open the side panel's activity log to show the classification decisions ("matches known product pattern", "asking AI...", "captured"). Visit a non-product page to show it being skipped / the domain blacklisted.
3. **Switch to the website (60s).** The just-captured items are already on the dashboard (Realtime). Show the auto-generated filter chips, the retailer rail, and sorting.
4. **Lists & sharing (45s).** Drop a few items into a list, set it to public, open the `/s/<id>` link in a private window to show the logged-out read-only view.
5. **The assistant (2 min).** Ask "which of my saved jackets is best for rain?" — show grounded answer + product card rail. Then "add the second one to my cart and favorite it" — show the action tags firing and the cart / Favorites updating live. Mention it works the same on a local Ollama model.
6. **Price tracking (45s).** Track a product, show the seeded history point, explain the daily cron + shared-row dedup design.
7. **Integrations (30s).** Connect eBay in the profile, show pulled order history. Mention the signed-state OAuth design.
8. **Architecture close (60s).** One slide: three clients, one Supabase, RLS as the security layer, BYO AI key, ~no custom server. Then the decisions in §7 that you want to highlight.
9. **Be upfront (20s).** Name two or three items from §9 so the audience knows you know.